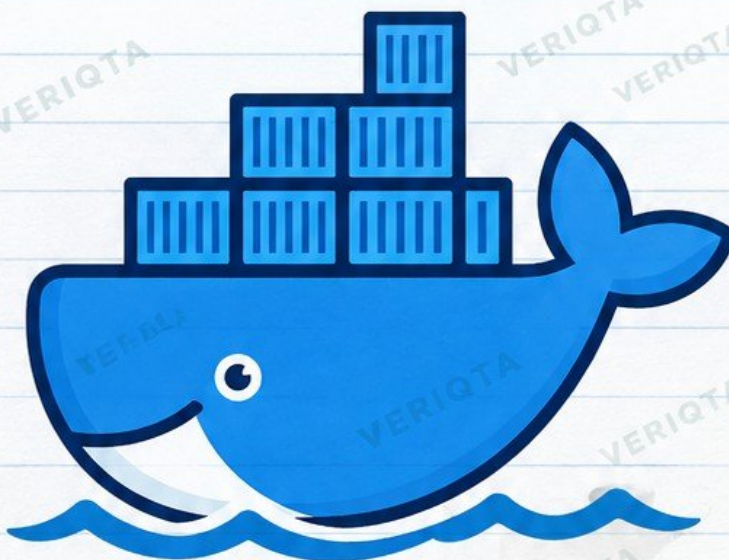


Docker

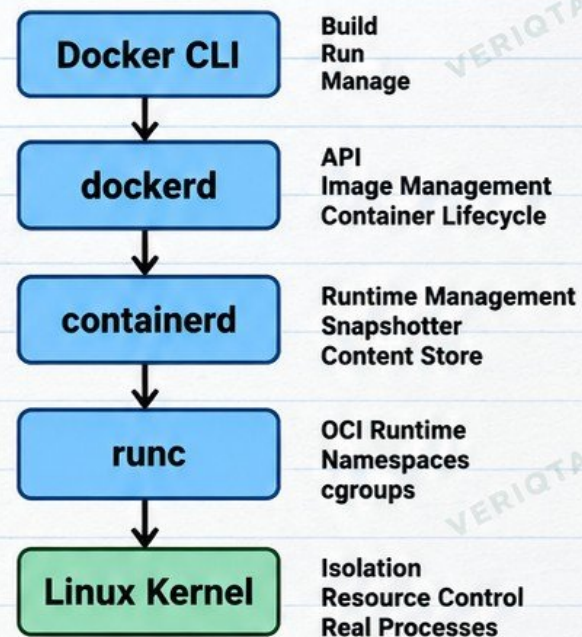
Under the Hood

A Deep Dive Into How Docker Really Works

20 - PAGE ENGINEERING NOTEBOOK



- ✓ Architecture
- ✓ Linux Internals
- ✓ Images & Storage
- ✓ Networking
- ✓ Resource Management
- ✓ Security
- ✓ Troubleshooting
- ✓ Real Production Insights



More Than
Commands
Understand
What's Really
Happening

From
Containers
to the Kernel

ENGINEER | DEBUG | OPTIMIZE | SCALE

Learn
Apply
Grow

VERIQTA

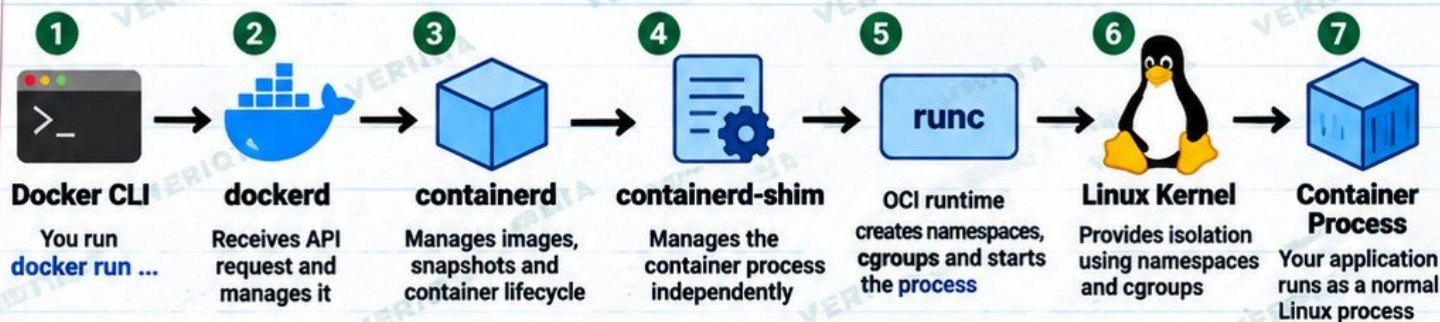
REAL ENGINEERING KNOWLEDGE

Better
Engineers
Build Better
Systems

What Actually Happens When You Run **docker run**

Same Command.
Real Process.

Trace the complete execution path



1 Docker CLI

- You run `docker run <image>`
- CLI formats the request and sends it to the Docker API
- Uses Unix socket `/var/run/docker.sock` (or TCP)

2 dockerd (Docker Daemon)

- Receives API request from the CLI
- Handles high-level logic (image resolution, network, volumes)
- Delegates container management to **containerd**

3 containerd

- Manages images, snapshots and content store
- Prepares the filesystem (snapshot) for the container
- Creates a container and starts a **containerd-shim**

4 containerd-shim

- A lightweight process that stays alive for the container
- Manages the container process lifecycle
- Allows the container to continue running even if **dockerd** stops

5 OCI Runtime (runc)

- containerd-shim** invokes **runc**
- runc** creates namespaces (PID, NET, MNT, UTS, IPC, USER)
- Sets up **cgroups** for resource control
- Executes the container process using the OCI spec

6 Linux Kernel

- Creates and isolates processes using namespaces
- Enforces resource limits using cgroups
- Provides filesystem isolation via OverlayFS
- Runs the container process as a normal Linux process

7 Container Process

- Your application runs inside the container
- It is a normal Linux process, isolated by namespaces
- It shares the just kernel (no separates operating system)

! Key Takeaway

- Docker does NOT create a miniature operating system.
- It uses Linux kernel features (namespaces, cgroups, OverlayFS)
- A container is just an isolated Linux process (or group of processes).
- Understanding this flow helps you debug issues at the right layer.

Full Execution Flow



Same Kernel.
More Possibilities.

Senior Insight

- The power (and risk) of containers comes from sharing the host kernel.
- When debugging, always know which layer you are dealing with: CLI, daemon, containerd, runtime or kernel.
- Most container problems are Linux problems.

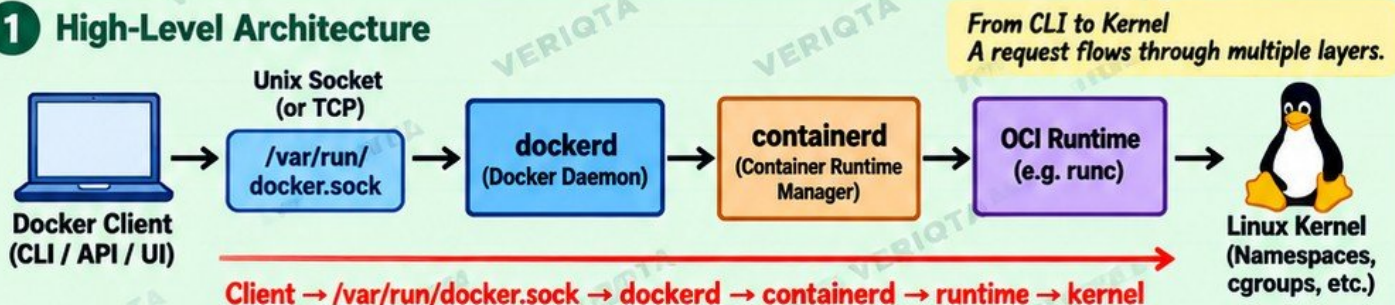
Learn
Build
Share
Grow

Docker Architecture Beyond the CLI

More Than Commands.
Understand the Engine.

Dissect Docker's control architecture.

1 High-Level Architecture



2 Client-Server Architecture

- Docker uses a client-server model.
- The Docker CLI is just a client.
- It sends requests to the Docker API.
- By default, it communicates over a Unix socket `/var/run/docker.sock` (or TCP if configured).
- You can also use tools like `curl` or SDKs to call the Docker REST API.

3 Docker API (REST)

- Docker exposes a REST API (e.g. `/containers`, `/images`, `/networks`).
- The CLI converts your command into an API request.
- API versioning ensures backward compatibility (e.g. v1.24, v1.41).
- You can query the API directly:

```
curl --unix-socket /var/run/docker.sock \
http://localhost/version
```

4 dockerd Responsibilities

- Listens for API requests.
- Authenticates and authorizes clients.
- Manages images, containers, networks and volumes.
- Delegates low-level tasks to containerd.
- Handles high-level orchestration and lifecycle management.
- Manages plugins and extensions.

5 containerd Responsibilities

- A high-performance container runtime manager (part of CNCF).
- Manages images, snapshots, content store.
- Handles container lifecycle (create, start, stop, delete).
- Uses `containerd-shim` to keep containers running independently.
- Supports multiple OCI runtimes (runc, crun, etc.).

6 OCI Runtime (runc)

- Implements the OCI Runtime Spec.
- Creates namespaces (PID, NET, MNT, UTS, IPC, USER).
- Sets up cgroups for resource control.
- Executes the container process.
- `runc` is a low-level tool. containerd invokes it.

7 Daemon Lifecycle

- `dockerd` runs as a long-lived process (daemon).
- Starts on system boot (systemd).
- Reads configuration from `/etc/docker/daemon.json`.
- Manages state in `/var/lib/docker/`.
- Can be restarted without stopping running containers (containerd-shim keeps them alive).

8 API Versioning

- Docker API is versioned (e.g. v1.24, v1.41).
- The CLI and server negotiate the highest mutual version.
- Helps maintain backward compatibility across releases.
- You can check the version with:

```
docker version
```

9 Privileged Daemon Implications

- The Docker daemon runs with high privileges (root).
- Access to `/var/run/docker.sock` allows full control of the host.
- Any user who can access the socket can start privileged containers, mount the host filesystem and potentially escape to the host.
- **Treat the Docker socket as a major security boundary.**

10 Daemonless Alternatives

- Podman (daemonless, rootless support)
- nerdctl (CLI for containerd)
- Buildah (build images without a daemon)
- These tools use the same underlying technologies (containerd, runc, OCI) but without a long-running daemon.
- Useful in restricted environments and for better security isolation.



Production Insight

- Access to the Docker socket (`/var/run/docker.sock`) is effectively highly privileged and must be treated as a major security boundary.

Secure the Socket
Secure the Host.

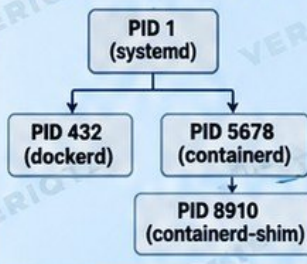
Containers Are Linux Processes

A container is not a mini OS. It's a group of Linux processes isolated by kernel features (namespaces, cgroups, etc).

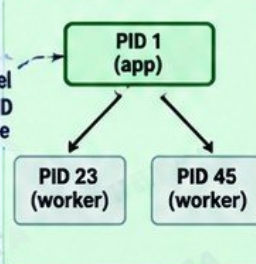
Same Kernel.
Same Processes.
Isolated Views.

1 What a Container Looks Like

Host (Real Linux System)



Container (Isolated View)



- Processes inside a container are normal Linux processes.
- They just run in different namespaces and cgroups.
- The kernel is shared with the host.
- PID numbers are different because of the PID namespace.

4 Important Concepts

- PID 1 inside containers**
The main process. If it doesn't handle signals or reap zombies, you can have problems.
- Parent-child relationships**
Same as any Linux system.
- Process visibility**
Isolated by PID namespace. Use `nsenter` to view from host.
- Signals**
PID 1 must handle SIGTERM, SIGINT, etc.
- Zombie processes**
Appear when child processes exit but are not reaped.
- Orphan processes**
Reparented to PID 1 inside the container namespace.
- Init processes**
Tools like `tini` or `dumb-init` help reap zombies and handle signals.
- Container exit semantics**
When PID 1 exits, the container stops (unless restarted).

2 Host PID vs Container PID

View	PID 1	Other PIDs	What You See
Host View	systemd (1)	432, 5678, 8910 ...	All processes (host + containers)
Container View	Application (1)	23, 45 ...	Only processes inside the container

! The same process has different PIDs in different namespaces. A container's PID 1 is not the host's PID 1.

3 /proc Inside a Container

- `/proc` shows process, memory, CPU and system information.
- Inside a container, you see the container's namespaced view.
- Some host information is hidden or limited.

```
# Inside the container
cat /proc/1/status
cat /proc/self/cgroup
cat /proc/mounts
```

/proc is a window into what the process sees, not the entire host system.

5 Useful Commands

Command	What It Does
<code>ps aux</code>	List processes inside the current namespace
<code>pstree -p</code>	Show process tree with PIDs
<code>docker top <container></code>	View running processes in a container
<code>docker inspect <container></code>	See PID, namespaces and other details
<code>nsenter -t <pid> -p -m -u -i -n</code>	Enter namespaces of a process from host

6 Failure Scenario: Zombie Processes

The application is healthy, but over time, zombie processes accumulate because PID 1 does not reap child processes.

```
$ ps aux
USER  PID  STAT  COMMAND
root   1    Ss    python app.py
root   23   Z     [worker] <defunct>
root   24   Z     [worker] <defunct>
root   25   Z     [worker] <defunct>
...
```

Why it happens:

- PID 1 doesn't call `wait()`
- Child processes exit
- They become zombies (Z)
- They are not reaped
- Zombies consume PIDs
- Can eventually exhaust the PID limit

7 How to Fix It

- ✓ Use a proper init process (`tini`, `dumb-init`)
- ✓ Ensure your application handles and reaps child processes
- ✓ Monitor for zombie processes
- ✓ Set a PID limit (`--pids-limit`)
- ✓ Test signal handling (`docker stop` sends SIGTERM)

Observe
Understand
Debug
Operate
Better

Remember:
Containers are just processes.
Good process management still matters.

Linux Namespaces: The Isolation Engine

Go beneath Docker's abstraction.

Same Kernel.
Different Views.

1 What Are Linux Namespaces?

- Namespaces isolate system resources so processes can have their own view of the system.
- A container is just a group of processes running in its own namespaces.
- Namespaced processes are **normal** Linux processes, using the **same kernel**.
- Namespaces provide **isolation**, not a separate operating system.

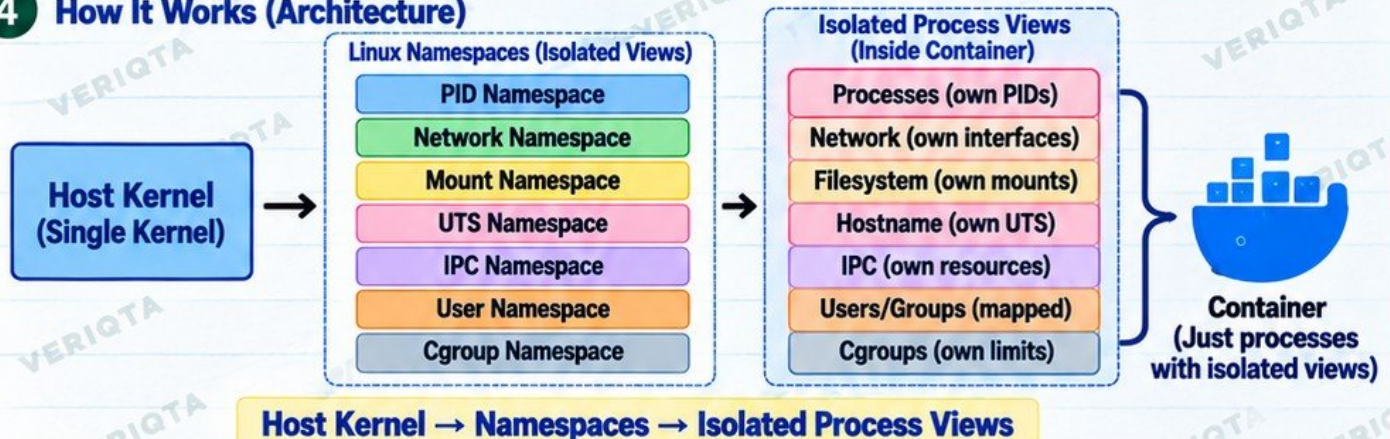
2 Types of Namespaces

Namespace	What It Isolates
PID	Process IDs (process tree)
NET	Network stack (interfaces, routing, ports, firewall)
MNT	Mount points (filesystem view)
UTS	Hostname and domain name
IPC	Inter-process communication (IPC objects)
USER	User and group IDs (UID/GID)
CGROUP	Cgroup hierarchy (resource view)

3 Key Benefits

- ✓ Process isolation
- ✓ Network isolation
- ✓ Filesystem isolation
- ✓ Independent hostnames
- ✓ Separate IPC resources
- ✓ Different user/group mapping
- ✓ Resource isolation via cgroups
- ✓ Better security and multi-tenancy

4 How It Works (Architecture)



Host Kernel → Namespaces → Isolated Process Views

5 Viewing Namespaces

Each process has namespace links under `/proc/<pid>/ns`

```
$ ls -l /proc/1234/ns
lrwxrwxrwx 1 root root 0 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 root root 0 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 root root 0 mnt -> 'mnt:[4026531840]'
lrwxrwxrwx 1 root root 0 net -> 'net:[4026531956]'
lrwxrwxrwx 1 root root 0 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 root root 0 user -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 uts -> 'uts:[4026531838]'
```

6 Entering Another Namespace

Use `nsenter` to run a command in a target process namespace.

```
# Enter the mount namespace of a container
sudo nsenter -t <pid> -m bash

# Enter the network namespace
sudo nsenter -t <pid> -n ip a

# Enter multiple namespaces
sudo nsenter -t <pid> -m -u -i -n -p bash
```

You need appropriate permissions (often root) to use `nsenter`.

7 Namespace Creation

Namespaces are created using `clone()` or `unshare()` system calls (used by runtimes).

```
# Create a new PID namespace
unshare -p --fork --mount-proc bash

# Create new network namespace
unshare -n bash
```

Container runtimes (runc, containerd) use these system calls when starting containers.

8 What Does Docker Actually Isolate?

- ✓ Process tree (PID namespace)
- ✓ Network stack (network namespace)
- ✓ Mount points (mount namespace)
- ✓ Hostname (UTS namespace)
- ✓ IPC resources (IPC namespace)
- ✓ User/Group IDs (user namespace, optional)
- ✓ Cgroups (resource isolation, not a namespace alone)
- ✗ It does **NOT** isolate the kernel, kernel modules, device drivers or system-wide resources.

Senior Question What does Docker actually isolate, and what remains shared?

9 Senior Engineer Takeaways

- Docker does not create a new operating system.
- All containers share the same Linux kernel.
- Namespaces isolate views, but are not a full security boundary.
- Understand namespace behavior to debug container issues.
- Combine namespaces with cgroups, capabilities, seccomp and LSMs for stronger isolation.

Namespaces give the illusion of a separate system, but it is still the same kernel underneath.

cgroups, How Docker Controls Resources

Explain resource governance at kernel level.

Control Resources.
Build Reliable Systems.

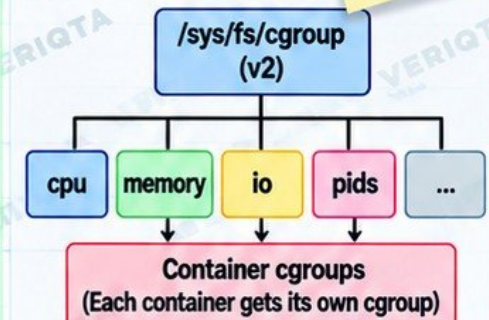
1 What Are cgroups?

- cgroups (control groups) let the Linux kernel limit, account for and isolate resource usage.
- Docker uses cgroups to control how much CPU, memory, I/O and other resources a container can use.
- cgroups do not create isolation like namespaces, they control resource consumption.

2 cgroups v1 vs v2

cgroups v1	cgroups v2
- Separate hierarchy per controller - Multiple mount points - More complex setup - Widely used (older systems) - Controllers: cpu, memory, blkio, etc.	- Unified hierarchy (single mount point) - Simplified model - Better resource management - Recommended for new systems - All controllers in one hierarchy

3 cgroup Hierarchy



4 Common Controllers

Controller	What It Controls
CPU	CPU share, quota, period, throttling
Memory	Memory limits, usage, OOM control
I/O (blkio)	Disk I/O bandwidth and IOPS
PIDs	Maximum number of processes
Others	cpuset, devices, hugetlb, etc.

5 Resource Accounting

- cgroups track resource usage for each container.
- You can view usage in: `/sys/fs/cgroup/`
- Key files (v2):
 - `cpu.stat`
 - `memory.current`
 - `memory.max`
 - `io.stat`
 - `pids.current`

The kernel enforces limits using cgroups, not Docker itself.

6 CPU Controls

- `--cpus` sets a CPU quota.
- `--cpuset-cpus` binds to specific CPUs.
- `--cpu-shares` sets relative weight (used during contention).

```
# Limit a container to 1.5 CPUs
docker run --cpus=1.5 nginx

# Set CPU shares
docker run --cpu-shares=512 nginx

# Check CPU quota inside container
cat /sys/fs/cgroup/cpu.max
```

7 Memory Controls

- `--memory` sets a memory limit.
- `--memory-swap` sets total memory + swap.
- When the limit is exceeded, the kernel can trigger OOM (Out of Memory) kill.



```
# Limit container to 512MB
docker run --memory=512m nginx

# Limit memory + swap
docker run --memory=512m --memory-swap=1g nginx

# Check memory usage
cat /sys/fs/cgroup/memory.current
cat /sys/fs/cgroup/memory.max
```

8 I/O and PIDs Controls

- I/O (blkio)
 - Limit disk read/write IOPS or bandwidth.
 - Useful for noisy neighbor control.

```
docker run --device-read-bps \
/dev/sda:10mb nginx
```

- PIDs
 - Limit the number of processes.
 - Prevents fork bombs and runaway processes.

```
docker run --pids-limit=100 nginx
```

9 Throttling and Limits

- If a container exceeds its CPU quota, the kernel throttles (pauses) the process.
- This can make the container look "slow" even when the host has free CPU.
- Check throttling metrics:

```
cat /sys/fs/cgroup/cpu.stat

# Look for:
nr_throttled
throttled_usec
```

Throttling protects the host, but it can cause latency and poor performance.

10 Production Failure Scenario

A container looks "slow" even though the host has available CPU.



- Cause: The container is hitting its CPU quota and being throttled.
- Symptoms: High latency, low CPU usage inside container, high `nr_throttled`.
- Fix: Increase `--cpus` or adjust CPU shares/limits based on workload.

11 Key Takeaways

- ✓ cgroups control resources, not isolation.
- ✓ cgroups v2 is the modern and simpler model.
- ✓ Use the right limits for CPU, memory, I/O and PIDs.
- ✓ Monitor resource usage and throttling metrics.
- ✓ Understand cgroup limits to avoid performance issues.

Memory Internals, OOM and Container Death

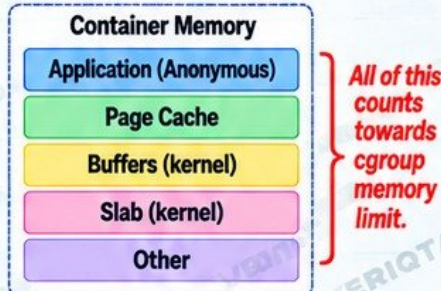
Deep dive into container memory behavior.

Understand
Memory.
Prevent
Outages.
Run Reliable
Containers.

1 Container Memory Components

- Anonymous Memory**
 - Memory used by applications (heap, stack, etc).
- Page Cache**
 - Cached file data from disk (improves performance).
- Working Set**
 - Actual memory actively used by the process.

2 Memory in a Container



3 cgroup Memory Accounting

- Docker uses cgroups to account and limit memory usage.
- Tracks anonymous memory, page cache, slab, etc.
- Enforces memory limits (no hard OS isolation, but processes can be killed).
- In cgroups v2, memory accounting is unified and more accurate.
- When the limit is exceeded, the kernel or cgroup OOM can kill the container.

4 Memory Limits and Swap

- `--memory` sets the cgroup memory limit.
- `--memory-swap` sets total memory + swap.
- If swap is not set, Docker may disable swap inside the container.
- When memory usage exceeds the limit, the kernel triggers OOM (Out of Memory).

```
# Limit container to 512MB
docker run --memory=512m nginx

# Set memory + swap
docker run --memory=512m --memory-swap=1g nginx
```

5 Kernel OOM vs cgroup OOM

Kernel OOM (host level)	cgroup OOM (container level)
- Triggered when the host is out of memory. - Kernel selects processes to kill (may kill containers). - Seen in dmesg.	- Triggered when a container exceeds its cgroup memory limit. - Only processes inside the container are killed. - Container stops with OOMKilled and exit code 137.

! OOMKilled = the container was killed due to out of memory. Exit code 137 = killed by SIGKILL.

6 Memory Pressure

- Memory pressure happens when available memory is low.
- The kernel reclaims page cache first.
- If pressure continues, the kernel starts killing processes.
- In containers, reaching the cgroup limit triggers cgroup OOM.
- This can happen even if the host still has free memory.

A container can be killed due to its own limit, even when the host has memory available.

7 Investigation Commands

Command	What It Shows
<code>docker stats</code>	Real-time memory usage (USAGE / LIMIT)
<code>docker inspect <id></code>	Memory limits and configuration
<code>dmesg grep -i oom</code>	Kernel OOM events (host level)
<code>cat /sys/fs/cgroup/memory.events</code>	cgroup OOM events (v2)
<code>cat /sys/fs/cgroup/memory.current</code>	Current memory usage (v2)
<code>cat /sys/fs/cgroup/memory.max</code>	Memory limit (v2)

8 Example: memory.events (cgroups v2)

```
$ cat /sys/fs/cgroup/memory.events
low      0
high     12
max       3
oom       3
oom_kill  3
```

oom and oom_kill increase when the container hits its memory limit.

9 Memory Leaks vs Legitimate Spikes

Memory Leak (Problem)	Legitimate Spike (Normal)
- Continuous increase over time. - Not released back to the OS. - Eventually hits limit and gets OOMKilled.	- Temporary increase due to workload (e.g. high traffic). - Memory returns to normal after spike. - Should be planned for in capacity limits.

Monitor trends, not just single spikes.

10 Real-World Scenario

A container looks "slow" even though the host has available CPU.

- !** Cause: The container is hitting its memory limit.
- Symptom: High latency, slow response, then OOMKilled (exit code 137).
- Investigation: Check `docker stats`, `memory.events` and `dmesg`.
- Fix: Increase memory limit or optimize application memory usage.

11 Key Takeaways

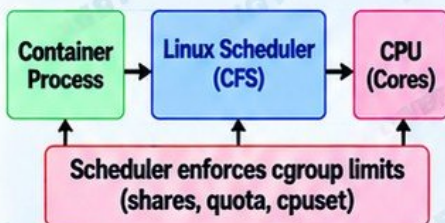
- Understand how anonymous memory, page cache and working set work.
- cgroups control and enforce memory limits.
- A container can be OOMKilled even if the host has free memory.
- Use `docker stats`, `docker inspect`, `dmesg` and `memory.events` for investigation.
- Distinguish between memory leaks and legitimate spikes.
- Plan memory limits based on real workload behavior.

Understand
CPU.
Fix
Performance.
Build
Reliable
Systems.

CPU Scheduling and Throttling Inside Containers

Explain why "50% CPU" can be misleading.

1 How It Works (Big Picture)



- Containers use the same Linux scheduler as host processes.
- Cgroups tell the scheduler how much CPU a container can use.
- The scheduler decides which process runs and for how long.

2 Key Concepts

Concept	What It Means
CPU shares	Relative weight when there is contention (default 1024).
CFS quota	Maximum CPU time allowed in a period.
CPU period	Time window for quota (default 100ms).
cpuset	Restricts container to specific CPUs (cores).
Throttling	Container is paused when it exceeds its quota.
Contention	Multiple workloads competing for the same CPU.
Multi-core	"50% CPU" can mean different things (per core or total).
Noisy neighbors	Other containers can affect available CPU time.

3 CFS Quota Example

Limit a container to 1 CPU (100ms period)

```
docker run --cpus=1 nginx
```

This sets:

- CPU quota = 100000 (100ms)
- CPU period = 100000 (100ms)

The container can run for 100ms every 100ms (1 full CPU).

4 CPU Shares Example

Set relative weight (default 1024)

```
docker run --cpu-shares=512 nginx
```

- If there is contention, this container gets 50% of the CPU time compared to a container with 1024 shares.
- Shares do not guarantee CPU, they only apply when CPUs are busy.

5 cpuset Example

Allow container to use specific CPUs

```
docker run --cpuset-cpus=0,1 nginx
```

- Restricts the container to CPU 0 and 1 (2 cores).
- Useful for performance-sensitive workloads.
- Prevents the scheduler from running the container on other CPUs.

6 Detecting Throttling

Check cgroup CPU statistics

```
# View CPU stats (cgroups v2)
cat /sys/fs/cgroup/cpu.stat
```

Key fields:

- `nr_throttled` → number of times throttled
- `throttled_usec` → total time throttled (in microseconds)

If `nr_throttled` and `throttled_usec` keep increasing, your container is hitting its CPU limit.

7 Interpreting CPU Usage

What You See	What It Can Mean
50% CPU	50% of 1 core, or 50% of all cores (depends on the tool)
100% CPU	1 full core (not necessarily all cores)
High load but low CPU %	Container is being throttled
Spikes in latency	CPU contention or throttling
Low CPU but slow application	Could be I/O, memory or external dependency

8 Common Causes of Throttling

- ✗ CFS quota set too low (`--cpus`)
- ✗ CPU contention with other containers (noisy neighbors)
- ✗ Misinterpreting CPU % in monitoring tools
- ✗ CPU-bound workloads hitting quota repeatedly
- ✗ High number of runnable processes
- ✗ Incorrect CPU limits for the workload

9 Real-World Incident

Scenario:
Application has high latency spikes even though average CPU utilization is 40–50%.

Root Cause:

- Container is being throttled (high `nr_throttled`).
- CPU quota is too low for the workload.

Impact: Requests take longer, timeouts, poor UX.

10 Fix and Optimize

- ✓ Increase CPU quota (`--cpus` or `--cpu-quota`)
- ✓ Adjust CPU shares based on priority
- ✓ Use `cpuset` for dedicated cores (if needed)
- ✓ Monitor `cpu.stat` for throttling
- ✓ Right-size limits based on real workload
- ✓ Identify and reduce noisy neighbors
- ✓ Use proper monitoring (per-core vs total CPU)

11 Key Takeaways

- ✓ Containers use the same Linux scheduler as the host.
- ✓ CPU shares control relative weight, not a hard limit.
- ✓ CFS quota and period enforce hard CPU limits.
- ✓ "50% CPU" can be misleading (per core vs total).
- ✓ Throttling can cause latency spikes even if average CPU looks fine.
- ✓ Use `cpu.stat` to detect throttling.
- ✓ Consider noisy neighbors and overall system contention.
- ✓ Tune CPU limits based on workload behavior and monitor continuously.

Docker Images Are Content-Addressed Filesystems

Dissect image structure.

Images
Are Immutable.
Build Smart.
Ship Faster.

1 What Is an OCI Image?

- Docker images follow the **OCI (Open Container Initiative)** image specification.
- An image is a set of content-addressed objects (manifest, config, layers).
- Images are immutable and built from read-only filesystem layers.
- Each object is identified by a **SHA-256 digest**, not by a name or tag.
- Tags (e.g. `nginx:latest`) are just human-friendly pointers.

2 Image Components

Component	Description
Manifest	Lists layers and the config object. Identifies the image.
Config Object	Contains metadata (env, cmd, labels, user, working dir, etc).
Filesystem Layers	Compressed, read-only layers that make up the root filesystem.
Layer Digests	Each layer has a SHA-256 hash (content address).
Image ID	Derived from the config digest.
Metadata	Includes architecture, OS, created time, etc.
Image Index	Points to multiple manifests for different platforms (multi-architecture).

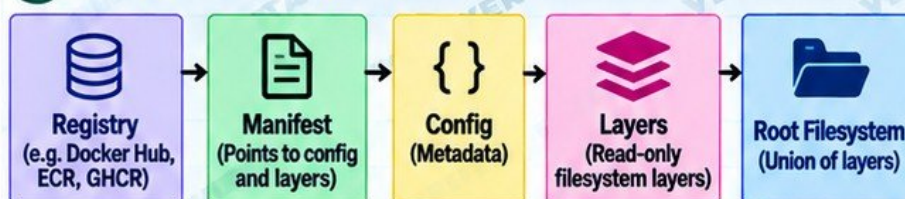
3 Content Addressing (SHA-256)

- Each layer and object is identified by a SHA-256 digest of its content.
- If the content changes, the digest changes.
- This ensures integrity, deduplication and immutability.
- Two images with the same content will have the same digest, even if the tag is different.

Example layer digest:

`sha256:3a6e0c8f2d4b7c9e...`
(64 hexadecimal characters)

4 Image Structure (Flow)



Registry → Manifest → Config → Layers → Root Filesystem

5 Multi-Platform Images

- An image **index** (manifest list) points to different manifests for different architectures (e.g. `linux/amd64`, `linux/arm64`).
- The correct image is pulled based on your platform.
- Use `docker buildx` to build multi-platform images.

```
# Build multi-platform image
docker buildx build --platform
linux/amd64,linux/arm64 -t myapp:latest
```

6 Inspecting an Image

```
# Inspect image details
docker image inspect nginx:latest

# View manifest (using skopeo or buildx)
skopeo inspect docker://nginx:latest

# View image history (layers)
docker history nginx:latest
```

7 Example: Image Layers

<code>sha256:a3f1...</code>	77MB	FROM ubuntu:22.04
<code>sha256:9b2c...</code>	5MB	RUN apt-get update
<code>sha256:4d8e...</code>	120MB	RUN apt-get install ...
<code>sha256:6f7a...</code>	2MB	COPY app/ /app
<code>sha256:1c9d...</code>	1MB	CMD ["python", "app.py"]

Each layer
is immutable
and identified
by its digest.

8 Why Changing One Dockerfile Instruction Invalidates Downstream Layers

- Each layer's digest is based on the content of that layer and all previous layers.
- When you change an instruction (e.g. `RUN`, `COPY`, `ADD`), the content changes.
- This creates a new digest for that layer, which invalidates all subsequent layers.
- Docker must rebuild the changed layer and every layer after it.
- This is why ordering instructions in your Dockerfile (least frequently changed first) improves build caching.

9 Key Takeaways

- ✓ Docker images are content-addressed (SHA-256).
- ✓ Images consist of manifest, config and layers.
- ✓ Layers are immutable and read-only.
- ✓ Image indexes enable multi-platform images.
- ✓ Changing one instruction invalidates downstream layers.
- ✓ Content addressing ensures integrity, deduplication and reproducibility.

1? Senior Engineer Question

How would you design a Dockerfile to maximize layer cache efficiency for a large application?

OverlayFS and the Container Writable Layer

Explain what happens to files inside a running container.

Containers Change Files. Images Stay the Same. Understand the Layers.

1 What is OverlayFS?

- OverlayFS is a union filesystem used by Docker to combine read-only image layers with a writable layer.
- It presents a single merged view of the filesystem inside the container.
- Files from image layers are read-only.
- Changes (create, modify, delete) are written to the container's writable layer (**upperdir**).

2 Key Components

Component	Description
lowerdir	Read-only image layers (multiple layers).
upperdir	Writable container layer (stores changes).
workdir	Used by OverlayFS for internal operations (copy-up, etc).
merged	The unified view presented inside the container.

3 How It Works (Simplified)

- When a file is read, OverlayFS looks in the **upperdir** first, then in the **lowerdir** layers.
- When a file is modified, it is copied from the **lowerdir** to the **upperdir** (copy-on-write) and then changed.
- When a file is deleted, a special **whiteout** file is created in the **upperdir** to hide the file in the lower layers.
- The **merged view** shows the final filesystem to the container.

4 OverlayFS Architecture

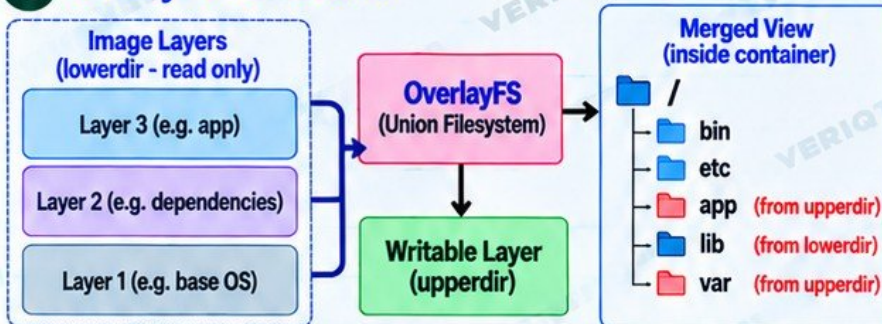


Image Layers + Writable Layer → Merged Container Filesystem

5 Copy-on-Write Example

- Original file in image layer (read-only)**

```
$ docker run --rm nginx cat /etc/nginx/nginx.conf
```

file comes from lowerdir (image layer)
- Modify the file inside the container**

```
$ docker run -it nginx bash
```

edit /etc/nginx/nginx.conf
The file is copied to upperdir and modified
- The modified file now exists in upperdir**
Changes are stored only in the container's writable layer, not in the image.

6 Whiteouts and File Deletion

- When a file is deleted inside the container, OverlayFS creates a **whiteout** file in the **upperdir**.
- The whiteout hides the file from lower layers in the merged view.
- The original file still exists in the image layer, but is not visible in the container.

Whiteout Example

```
$ ls -la /var/lib/docker/overlay2/<id>/diff/etc
-rw-r--r-- 1 root root 0 .wh.nginx.conf
```

.wh.<filename>
indicates a whiteout

7 Important Concepts

Read-only layers	Image layers cannot be changed.
Writable layer	All changes are stored here.
Copy-on-write	Modified files are copied to upperdir before being changed.
Whiteouts	Special files that hide deleted files.
Layer lookup	OverlayFS checks upperdir first, then lowerdir layers.
Write amplification	Small changes can result in large copy operations (due to copy-up).

8 Inspecting OverlayFS

- Check the storage driver:

```
$ docker info | grep "Storage Driver"
```

Storage Driver: overlay2
 - Inspect mount details:

```
$ mount | grep overlay
```
 - Look at the container's upperdir:

```
$ docker inspect <container_id> | grep -A 5 "UpperDir"
```
- Paths are usually under:
/var/lib/docker/overlay2/

9 Performance Insight

- The writable layer is designed for container changes, not as a database or application storage.
- Heavy write workloads can be slower due to copy-on-write and write amplification.
- Use volumes or bind mounts for databases, logs and high I/O workloads.
- The container writable layer is not a substitute for properly designed persistent storage.

10 Key Takeaways

- OverlayFS combines read-only image layers with a writable layer. Changes are stored in the upperdir using copy-on-write.
- File deletions use whiteout files.
- Understanding the merged view explains what you see inside the container.
- Use volumes for persistent and high-write workloads.
- Container writable layers are temporary and not a replacement for persistent storage.

Docker Build Internals and BuildKit

Go behind docker build.

Better Builds.
Faster CI.
Smarter Engineers.

1 Dockerfile Parsing

- Docker reads the Dockerfile line by line.
- Each instruction is converted into an internal representation.
- With BuildKit, the Dockerfile is parsed into a **Low-Level Build (LLB) graph**.
- The parser understands instructions, dependencies and context to build efficiently.

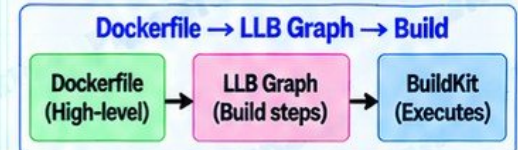
2 What is BuildKit?

- BuildKit is the modern build engine for Docker.
- It builds a graph of build steps (**LLB**).
- Supports parallel execution, better caching, secret mounts, and more.
- Enabled by default in modern Docker (Docker 23+).
- Use `DOCKER_BUILDKIT=1` to enable if needed.

```
# Enable BuildKit
export DOCKER_BUILDKIT=1
docker build .
```

3 Build Graph and LLB

- Dockerfile is converted to a **Low-Level Build (LLB) graph**.
- LLB represents all build steps and their dependencies.
- BuildKit can run independent steps in parallel.



4 Build Context

- The build context is the set of files sent to the Docker daemon.
- A large context can slow down builds.
- Use `.dockerignore` to exclude unnecessary files.

```
# Example .dockerignore
node_modules/
.git/
*.log
Dockerfile
.env
```

Smaller context = faster builds.

5 Cache Keys and Layer Caching

- Each instruction generates a cache key based on:
 - Instruction text
 - Files it depends on
 - Build context content
 - Previous layer state
- If the cache key matches, the layer is reused.
- If anything changes, the cache is invalidated and the layer must be rebuilt.

6 Cache Invalidation Example

- Changing a file used in a **COPY** instruction invalidates that layer and all subsequent layers.
- Place instructions from least changing to most changing.

Bad (Slow Builds)	Good (Better Caching)
<code>COPY . .</code>	<code>COPY package*.json ./</code> <code>RUN npm install</code> <code>COPY . .</code>

7 Parallel Execution

- BuildKit can run independent steps in parallel, reducing build time.
- It analyzes the build graph to find steps that do not depend on each other.
- This is especially useful in multi-stage builds.

8 Multi-Stage Builds

- Use multiple **FROM** stages.
- Build artifacts in one stage and copy only what you need to the final image.
- Results in smaller, more secure images.

```
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
```

9 Build Mounts and Secret Mounts

Build Mounts	Secret Mounts
• Mount files/directories during build without adding them to the image. • Useful for caches (e.g. package managers). <pre>RUN --mount=type=cache, target=/root/.npm npm install</pre>	• Use secrets (e.g. API keys) during build without baking them into the image. • Keep sensitive data secure. <pre>RUN --mount=type=secret, id=api_key cat /run/secrets/api_key</pre>

10 Remote Cache

- Store and reuse build cache in a remote registry (e.g. GitHub Actions, Docker Hub, GitLab).
- Speeds up CI/CD builds.

```
docker buildx build \
  --cache-from=type=registry,ref=org/app:cache \
  --cache-to=type=registry,ref=org/app:cache,mode=max \
  -t org/app:latest .
```

11 Reproducibility

- Use pinned base image versions.
- Keep builds deterministic.
- Avoid unnecessary files in context.
- Use lock files (package-lock.json, etc.).
- Ensures the same input produces the same output.

Reproducible builds = reliable systems.

12 Real-World Failure Scenario

A poorly positioned **COPY** instruction destroys useful caching and turns a small build into a slow CI bottleneck.

- **Cause:** `COPY . .` early in the Dockerfile.
- **Result:** Every code change invalidates the cache.
- **Impact:** Long build times, slow deployments, wasted CI resources.
- **Fix:** Copy files in stages (dependencies first), use `.dockerignore` and take advantage of BuildKit.

13 Key Takeaways

- ✓ BuildKit uses an LLB graph to build images efficiently.
- ✓ Cache keys determine whether a layer can be reused.
- ✓ Order your Dockerfile instructions to maximize caching.
- ✓ Use multi-stage builds, build mounts and secret mounts.
- ✓ Leverage remote cache for faster CI/CD.
- ✓ Small changes can have a big impact on build performance.

Build Smart.
Cache More.
Ship Faster.

VERIQTA

REAL ENGINEERING KNOWLEDGE



Learn Build Share Grow

From Registry Pull to Local Image

Trace an image across the network.

Same Image.
Same Layers.
Different
Tags.
Know What
You Deploy !

1 Registry API

- Docker uses the OCI Registry API to communicate with registries (Docker Hub, ECR, GHCR, etc).
- The client sends HTTP requests to fetch manifests and blobs.
- The API is standard, so different registries work the same way.
- Endpoints: /v2/, /manifests/, /blobs/

2 Authentication

- Authenticate using username/password or an access token (e.g. docker login).
- The client receives a token (Bearer token) and uses it for subsequent requests.
- Tokens have scopes (pull, push, etc).
- Example:

```
docker login
docker pull nginx:latest
```

3 Repository and Tags

- A repository is a collection of images (e.g. library/nginx).
- A tag (e.g. latest, 1.25) is a human-friendly label that points to a manifest.
- Tags are mutable (they can change).
- Use tags for convenience, but understand they are not immutable.

4 Manifest Retrieval

- The client requests the manifest for the specified tag.
- The registry returns a manifest (JSON) that contains:
 - Image configuration
 - List of layer digests
 - Media types
 - Architecture/platform info

5 Digest Verification

- Each layer and the manifest have a SHA-256 digest.
- The client verifies the digest to ensure the content is not corrupted or tampered with.
- If the digest does not match, the download fails.

6 Blob Download

- The client downloads each layer (blob) from the registry using its digest.
- Layers are downloaded in parallel.
- If a layer already exists locally, it is skipped (deduplication).

7 Decompression and Storage

- Downloaded layers are compressed (usually gzip).
- The client decompresses and stores them in the local content store.
- Layers are stored by their content digest, not by tag.

8 End-to-End Flow



Tag → Registry → Manifest → Digests → Blobs → Local Content Store

9 Layer Deduplication

- If a layer already exists locally (same digest), it is not downloaded again.
- Different images can share layers (saves bandwidth and disk space).
- This is why pulling similar images is faster.

10 Tag vs Digest

Tag	Digest
Human-friendly (e.g. latest, 1.25)	Immutable (e.g. sha256:abcd...)
Can change over time	Never changes
Good for development	Recommended for production
Points to a manifest	Points to a specific manifest

11 Multi-Platform Images

- A tag can point to a manifest list (index) for multiple architectures (linux/amd64, linux/arm64, etc).
- The client pulls the correct image for your platform.
- Each platform has its own manifest and layers, but shares common layers when possible.

12 Local Content Store

- Layers and manifests are stored in /var/lib/docker (or containerd).
- Stored by content digest.
- Multiple images can reference the same layers.
- Tags are just pointers to manifests, not the actual data.

13 Security Focus

- Using :latest in production is risky because the tag can change without notice.
- Always prefer immutable digests for production deployments (e.g. nginx@sha256:abcd...).
- This ensures you deploy the exact same image every time.
- Helps with security, reproducibility and compliance.

14 Real-World Example

```
# Pull by tag (mutable)
docker pull nginx:latest

# Pull by digest (immutable)
docker pull nginx@sha256:3a6e0c8f2d4b7c9e...

# Inspect image
docker inspect nginx:latest
```

15 Key Takeaways

- ✓ Understand the registry API flow.
- ✓ Tags are mutable, digests are immutable.
- ✓ Docker verifies content using SHA-256.
- ✓ Layers are deduplicated and cached locally.
- ✓ Use immutable digests for production.
- ✓ Same image, different tags can point to different content.

Container Networking From Namespace to Internet

Trace a packet completely.

Networking
Is Not Magic.
It's Linux.
Understand
It.
Control It.

1 Network Namespaces

- Each container has its own network namespace with its own network stack (interfaces, routes, iptables).
- Provides isolation from the host and other containers.
- The container sees its own `eth0`, routing table and default gateway.

2 Veth Pairs

- A veth pair is a virtual Ethernet cable connecting the container namespace to the host namespace.
- One end (e.g. `eth0`) lives inside the container.
- The other end (e.g. `vethXXXX`) lives on the host and is attached to a bridge (e.g. `docker0`).

3 Docker Bridge

- Docker creates a Linux bridge (`docker0`) on the host.
- The host side of each veth pair is connected to this bridge.
- The bridge allows containers on the same network to communicate.
- It also connects to the host's routing and NAT for external access.

4 Container and Host Interfaces

Inside Container (namespace)

```
# ip a
1: lo
2: eth0@if123
   inet 172.17.0.2/16
   (container IP)
```

- `eth0` is the container's interface.
- It has an IP from the Docker subnet (e.g. `172.17.0.0/16`).
- Default gateway is the bridge IP (e.g. `172.17.0.1`).

On Host (namespace)

```
# ip a
3: docker0
   inet 172.17.0.1/16
4: veth1234@if5
   (connected to container)
```

- `docker0` is the bridge interface.
- `vethXXXX` is the host side of the veth pair.
- The host routes traffic between the bridge and external networks.

5 Routing and Default Gateway

- The container sends traffic to its default gateway (`docker0`).
- The host routes the traffic to other containers or to the internet.
- Use `ip route` inside the container to view its routing table.

```
# ip route (inside container)
default via 172.17.0.1 dev eth0
172.17.0.0/16 dev eth0
```

```
# ip route (on host)
172.17.0.0/16 dev docker0
...
```

6 ARP and Neighbour Discovery

- Containers use ARP to find the MAC address of the default gateway (`docker0`).
- The bridge responds with its MAC address.
- This allows the container to send the packet to the host through the veth pair.

```
# arp -n (inside container)
? (172.17.0.1) at 02:42:ac:11:00:01 [ether] on eth0
```

7 NAT and Masquerading

- The host uses NAT (masquerading) to allow containers to access external networks.
- Source IP (container IP) is translated to the host's IP.
- Implemented using iptables (or nftables).

```
# iptables -t nat -L -n
Chain POSTROUTING
MASQUERADE all -- 172.17.0.0/16 !docker0
```

8 Port Publishing

- Maps a host port to a container port using DNAT.
- Allows external access to services inside containers.
- Example: `-p 8080:80`

```
# iptables -t nat -L -n | grep 8080
DNAT tcp -- 0.0.0.0/0 0.0.0.0/0
tcp dpt:8080 to:172.17.0.2:80
```

9 Conntrack (Connection Tracking)

- Linux tracks the state of network connections.
- Ensures return traffic is correctly routed back to the container.
- Helps with NAT, port publishing and firewall rules.

```
# conntrack -L | head
tcp        6 431999 ESTABLISHED
src=172.17.0.2 dst=8.8.8.8
...
```

10 Packet Path (End-to-End Flow)



Container `eth0` → veth → `docker0` → Host Routing/NAT → Host NIC → Network

11 Key Takeaways

- Containers use network namespaces.
- veth pairs connect containers to the host.
- `docker0` is a Linux bridge.
- NAT (masquerading) enables internet access.
- Port publishing uses DNAT.
- Conntrack maintains connection state.
- Understand the full packet path.
- Docker networking is standard Linux networking with namespaces, bridges and iptables — not magic.

12 Make It Real (Hands-On)

- Create a container and inspect its network:

```
docker run -d --name web nginx
ip a (inside container)
iptables -t nat -L -n (on host)
conntrack -L | grep <container-ip>
```

13 Security Focus

- Understand what ports are exposed.
- Use specific port mappings (not `0.0.0.0`).
- Restrict access with firewall rules.
- Be aware of container breakout risks if the host is compromised.
- Monitor network traffic and connections.

Docker DNS and Service Discovery Internals

Explain name resolution.

Names Connect Containers. Understand The DNS Behind It.

1 /etc/resolv.conf in Containers

- Each container has its own `/etc/resolv.conf`.
- For user-defined networks, Docker sets the nameserver to its embedded DNS (127.0.0.11).
- This file may also include search domains and options.

```
# Inside a container
cat /etc/resolv.conf

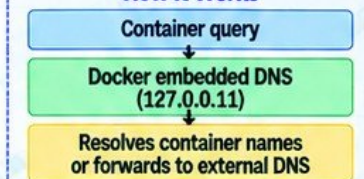
nameserver 127.0.0.11
options ndots:0
search myapp_default
```

Docker uses 127.0.0.11 for internal DNS resolution.

2 Embedded Docker DNS

- Docker runs an embedded DNS server at 127.0.0.11.
- It resolves container names and network aliases to IP addresses within the same user-defined network.
- It also forwards external queries to the host's DNS servers.

How It Works



3 User-Defined Networks

- Automatic DNS-based service discovery is available on user-defined networks (e.g. bridge, overlay).
- Containers on the same network can resolve each other by name.
- The default bridge network (docker0) does not provide automatic name resolution.

```
# Create a user-defined network
docker network create myapp
```

```
# Run containers on the same network
docker run -d --name web --network myapp nginx
docker run -d --name db --network myapp postgres
```

Use user-defined networks for service discovery.

4 Container-Name Resolution

- Containers can resolve each other using the container name.
- You can also define network aliases for alternative names.
- Resolution is scoped to the network the containers share.

```
# Run a container with an alias
docker run -d --name db \
  --network myapp --network-alias database \
  postgres

# From another container
ping db
ping database
```

Same network = Name resolution works.

5 DNS Forwarding

- If a name is not found in Docker's internal records, the embedded DNS forwards the query to the host's DNS servers.
- This allows containers to resolve external domains (e.g. google.com).

```
# Check external resolution
ping google.com

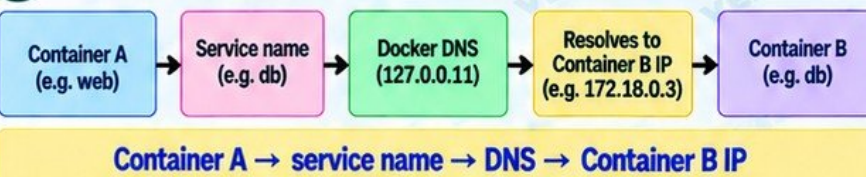
# View DNS configuration
cat /etc/resolv.conf
```

6 DNS Caching Behavior

- Docker's embedded DNS has a short TTL (typically 600 seconds).
- Applications inside containers may also cache DNS results (e.g. JVM, Go, Node.js).
- This can lead to stale IPs if containers are recreated or scaled.
- Do not assume DNS changes are seen immediately by your application.

DNS Changes may take time to propagate in your app.

7 End-to-End Resolution Flow



9 Debugging Name Resolution

- Check `/etc/resolv.conf`.
- Use `nslookup` or `dig` to test resolution.
- Verify both containers are on the same network.
- Check network aliases.
- Test direct IP connectivity.

```
# Install tools (if needed)
apt-get update && apt-get install -y dnsutils

# Test resolution
nslookup db
dig db

# Check network
docker network inspect myapp
```

10 Common Pitfalls

- Using the default bridge network (no automatic DNS).
- Containers on different networks.
- Typos in container names or aliases.
- Assuming DNS changes are instant.
- Application-level DNS caching.
- Hardcoding IP addresses instead of using service names.

Use service names, not IPs.
Build reliable, scalable applications.

8 Real-World Incident

Scenario:

- Container A cannot resolve the service name for Container B, but direct IP connectivity works.

Possible causes:

- Containers are on different networks.
- Wrong network alias or name.
- DNS caching in the application.
- Application using a custom DNS resolver.
- Misconfigured `/etc/resolv.conf`.

11 Key Takeaways

- Each container has its own `/etc/resolv.conf`.
- Docker provides an embedded DNS (127.0.0.11) for user-defined networks.
- Containers can resolve each other by name.
- DNS queries are forwarded for external domains.
- Understand DNS caching behavior.
- Use network aliases for flexibility.
- Don't rely on immediate DNS updates.
- Always prefer service names over IPs.
- Proper network design avoids resolution issues.

Port Publishing, NAT and Connection Tracking

Go deeper than -p 8080:80.

Understand
the Network.
Debug
Faster.
Be a Better
Engineer.

1 What is Port Publishing?

- Port publishing maps a port on the host to a port on the container.
- Syntax: `-p <host_port>:<container_port>`
- It uses **DNAT** (Destination NAT) and `iptables/nftables` to forward traffic.
- It allows external clients to access services running inside a container.

```
# Example
docker run -d -p 8080:80 nginx
# Host port 8080 → Container port 80
# Traffic to host:8080 is forwarded to the container.
```

2 Listening Sockets

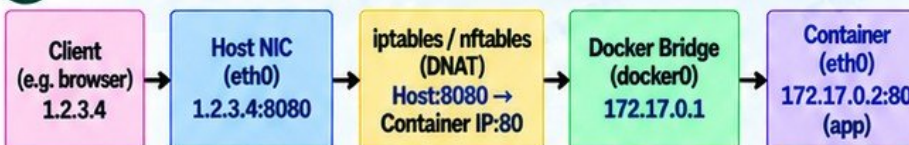
- The application inside the container must listen on the container port (e.g. `0.0.0.0:80`).
- If the app only listens on `127.0.0.1`, external traffic will not reach it.

```
# Check listening sockets (inside container)
ss -tln
# Or
netstat -tln
```

3 Host Ports vs Container Ports

- Host port: port on the host machine (e.g. 8080).
- Container port: port inside the container (e.g. 80).
- Host port can be different from container port.
- Multiple containers cannot bind to the same host port (port collision).

4 How It Works (Packet Path)



Client → Host Port → DNAT → Forwarding → Container

5 DNAT, Forwarding and Masquerading

- DNAT**: changes the destination IP/port to the container's IP/port.
- Forwarding**: allows the packet to be routed to the container through `docker0`.
- Masquerading (SNAT)**: changes the source IP for outgoing traffic from containers to external networks.
- Implemented using `iptables` or `nftables` rules created by Docker.

6 Connection Tracking (conntrack)

- Linux tracks all connections in the `conntrack` table.
- It keeps the state of each connection (NEW, ESTABLISHED, RELATED, INVALID).
- This allows return traffic to be sent back correctly.
- You can inspect `conntrack` entries with:

```
# View connection tracking entries
sudo conntrack -L | grep 8080
```

conntrack
remembers
connections
so return traffic
works.

7 Ephemeral Ports

- When a client connects, it uses a random high port (ephemeral port) as the source port.
- Default Linux ephemeral port range: 32768 – 60999
- Important for firewalls and troubleshooting.

```
# Check ephemeral port range
cat /proc/sys/net/ipv4/ip_local_port_range
```

8 Binding to 0.0.0.0 vs localhost

- `-p 8080:80` (default) binds to `0.0.0.0:8080` (all interfaces).
- `-p 127.0.0.1:8080:80` binds only to localhost (not accessible from other machines).
- Use localhost binding for local development and better security.

9 IPv4 / IPv6 Considerations

- Docker uses IPv4 by default (bridge network).
- IPv6 is supported but requires additional configuration (`daemon.json`).
- Be aware of dual-stack environments and firewall rules.

10 Security Consequences

- Publishing ports exposes services to the network.
- Avoid exposing unnecessary ports.
- Use firewalls (UFW, `iptables`, cloud security groups).
- Prefer binding to `127.0.0.1` for internal services.
- Be careful with `0.0.0.0` in production.

11 Real-World Example

Check port mapping and rules:

```
# Docker port mapping
docker ps
# iptables rules (DNAT)
sudo iptables -t nat -L -n | grep 8080
# conntrack entry
sudo conntrack -L | grep 8080
```

12 Common Issues

- Port already in use (collision).
- Container is running but app is not listening on the correct port.
- Firewall blocking the port.
- Wrong bind address (`127.0.0.1` vs `0.0.0.0`).
- Missing or incorrect DNAT rules.
- Connection succeeds but no response (application issue).

13 Key Takeaways

- ✓ Understand the full packet path.
- ✓ Know the difference between host port and container port.
- ✓ Check listening sockets inside the container.
- ✓ Use `conntrack` to troubleshoot connection issues.
- ✓ Be mindful of security when exposing ports.
- ✓ Prefer localhost binding for non-public services.
- ✓ Always verify firewall and network rules.

Know
The Packet Path.
Solve Problems.
Build Secure
Systems.

Container Storage, Volumes and Bind Mounts

Explain storage below Docker abstractions.

Data Persists. Containers Are Ephemeral. Plan Properly.

1 Container Writable Layer

- Each container has a thin writable layer on top of read-only image layers.
- Changes are stored using OverlayFS (Copy-on-Write).
- Data is lost when the container is removed.
- Not suitable for persistent data (databases, uploads, etc).

```
# View container writable layer size
docker diff <container>
docker inspect <container> --format '{{.GraphDriver}}'
```

2 Named Volumes

- Managed by Docker and stored in `/var/lib/docker/volumes/`.
- Independent of container lifecycle.
- Easier backup, sharing and reuse across containers.
- Best for persistent application data.

```
# Create and use a named volume
docker volume create mydata
docker run -d -v mydata:/data nginx
# List volumes
docker volume ls
# Inspect volume
docker volume inspect mydata
```

3 Bind Mounts

- Mounts a host directory or file into the container.
- Uses a specific path on the host.
- Changes on the host are immediately visible in the container (and vice versa).
- Useful for development and configuration files.
- Tightly coupled to the host filesystem.

```
# Bind mount example
docker run -d -v /host/path:/app nginx
# Read-only bind mount
docker run -d -v /host/path:/app:ro nginx
```

4 tmpfs Mounts

- Stores data in memory (RAM).
- Data is lost when the container stops.
- Useful for temporary data, caches, sensitive files.
- Improves performance and security for short-lived data.

```
# tmpfs example
docker run -d --tmpfs /cache \
--tmpfs /run nginx
```

5 Mount Propagation

- Controls how mounts are shared between host and containers.
- Modes: `rprivate` (default), `rshared`, `rslave`, `runbindable`.
- Important when running nested containers or Kubernetes (Docker-in-Docker).

6 Ownership and UID/GID

- Files keep their UID/GID from the host.
- Permission issues can occur if the container user does not match host ownership.
- Use `--user`, `chown`, or matching UID/GID.

7 Filesystem Semantics

- Container storage uses the host filesystem (ext4, xfs, etc).
- Certain features (inodes, file watching, locking) may behave differently.
- Performance depends on the storage driver and underlying filesystem.

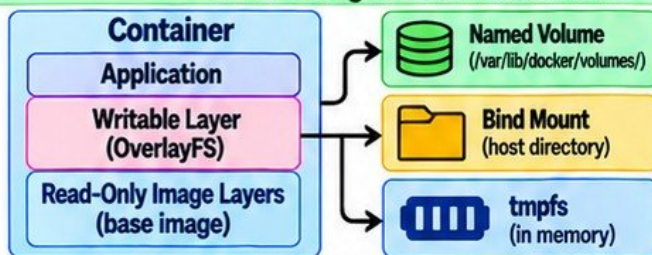
8 Persistence and Backup

- Writable layer: not persistent.
- Named volumes: persistent, easy to backup.
- Bind mounts: persistent, but tied to host.
- tmpfs: not persistent.
- Plan backups for important data.

9 Host Dependency

- Bind mounts depend on host directory structure.
- Moving containers to another host may break bind mounts.
- Named volumes are more portable (Docker manages the location).

How Storage Works in a Container



Key Points

- Containers are ephemeral, data must be stored outside.
- Choose the right storage type for your use case.
- Understand permissions and UID/GID mapping.
- Be aware of host dependency (especially with bind mounts).
- Plan backup and recovery for important data.
- Consider security implications (read-only mounts, least privilege).

Trade-Off: Writable Layer vs Volume vs Bind Mount vs tmpfs

Feature	Writable Layer	Named Volume	Bind Mount	tmpfs
Persistence	No (lost on container removal)	Yes	Yes (on host)	No (in memory only)
Performance	Good (but limited size)	Good	Depends on host filesystem	Very fast
Portability	Low (container specific)	High	Low (host dependent)	Low
Security	Medium	High (managed by Docker)	Lower (direct host access)	High (not written to disk)
Operational Control	Low	High	High (full host control)	Medium
Use Cases	Temporary data	Databases, persistent data	Development, config files	Caches, sensitive data



Real-World Example

- You are running a database in a container. If you store data in the writable layer and the container is removed, you will lose the data. Use a named volume or bind mount to persist the data.



Senior Takeaway

- Understand the storage options and their trade-offs.
- Choose the right type based on persistence, performance, portability, security and operational needs.
- Storage design is critical for production reliability.

Learn
Build
Share
Grow

Container Startup, PID 1, Signals and Shutdown

Trace the complete lifecycle.

Graceful Shutdown Saves Data. Understand PID 1. Build Reliable Systems.

1 Container Creation

- You run `docker run` (or `create`).
- Docker uses a runtime specification (OCI) to create the container.
- Namespaces, cgroups, filesystem, and network are set up.
- The container is created in a stopped state, then started with the specified command (ENTRYPOINT + CMD).

2 ENTRYPOINT and CMD

- ENTRYPOINT**: the main command that always runs.
- CMD**: default arguments for the command.
- CMD can be overridden at runtime.
- Common pattern: use ENTRYPOINT for the application and CMD for default arguments.

3 Exec Form vs Shell Form

- Exec form (recommended): `["nginx", "-g", "daemon off;"]`
- Runs the process directly. Proper signal handling.
- Shell form: `nginx -g "daemon off;"`
- Runs through a shell (e.g. `/bin/sh -c`). Signals may not be forwarded properly.

4 PID 1 in Containers

- The main process inside the container runs as PID 1.
- PID 1 has special behavior in Linux. It ignores some signals by default and must properly handle and forward signals to child processes.
- If your application is not PID 1, use a minimal init system (e.g. `tini`) or ensure proper signal forwarding.

5 Signal Handling

- Docker sends `SIGTERM` to PID 1 when stopping a container (`docker stop`).
- The application should catch `SIGTERM` and shut down gracefully (close connections, finish requests, flush data).
- If PID 1 does not forward signals, child processes may never receive them.

6 Grace Period and SIGKILL

- Docker waits for a grace period (default 10 seconds) after sending `SIGTERM`.
- If the container is still running after the grace period, Docker sends `SIGKILL`.
- `SIGKILL` cannot be caught or ignored.
- This may cause request failures and data loss.

7 Exit Codes

- 0: success
- 1: general error
- 137: killed by `SIGKILL` (128 + 9)
- Helps in debugging container failures.

8 Restart Policies

- `no`: do not restart (default)
- `on-failure`: restart when exit code is non-zero
- `always`: always restart
- `unless-stopped`: always restart unless manually stopped
- Useful for resilience, but understand the root cause of crashes.

9 Complete Container Lifecycle



10 Real-World Failure Scenario



Common causes:

- Using shell form instead of exec form.
- PID 1 not forwarding signals.
- Application does not handle `SIGTERM`.
- Grace period too short.
- Long-running requests not finishing.
- No proper shutdown logic (closing DB connections, flushing data, etc.).

If your app never receives `SIGTERM`, Docker will `SIGKILL` it. Plan for graceful shutdown.

13 Key Takeaways

- Understand the full container lifecycle from creation to shutdown.
- Use exec form and proper signal handling.
- PID 1 matters.
- Test graceful shutdown to avoid data loss.
- Use appropriate restart policies and monitor exit codes.

11 Best Practices

- Use exec form for ENTRYPOINT.
- Ensure PID 1 handles and forwards signals.
- Implement graceful shutdown in your application.
- Give enough grace period for shutdown (`--stop-timeout`).
- Use a minimal init system (e.g. `tini`) if needed.
- Test signal handling in a real container.
- Monitor exit codes and container logs.

12 Useful Commands

```

# Run a container
docker run --name app myimage
# Stop a container (sends SIGTERM)
docker stop app
# Set custom stop timeout
docker stop -t 30 app
# Check container logs
docker logs app
# Inspect container config
docker inspect app
  
```

Docker Security Boundaries Under the Hood

Explain what actually restricts a container.

Containers
Are Isolated
Processes,
Not Mini VMs.
Know The
Limits.

1 Linux Capabilities

- Linux breaks the all-powerful root into fine-grained capabilities.
- Containers run with a reduced set of capabilities by default.
- You can add or drop capabilities using `--cap-add` or `--cap-drop`.

Example: add a capability

```
docker run --cap-add=NET_ADMIN
myimage
```

2 seccomp

- seccomp filters which syscalls a container can make.
- Default Docker seccomp profile blocks dangerous syscalls.
- Helps prevent container escape and kernel attacks.

View default seccomp profile

```
cat /etc/docker/seccomp.json
```

3 AppArmor

- Linux Security Module (LSM) that restricts what processes can do.
- Uses profiles to control file access, capabilities and system resources.
- Enabled by default on many Linux distributions (e.g. Ubuntu).

Check AppArmor profile

```
docker inspect --format
'{{.AppArmorProfile}}'
<container>
```

4 SELinux

- Another LSM that enforces mandatory access control (MAC).
- Helps isolate containers from the host and other containers.
- Common on RHEL/CentOS/Fedora systems.

Check SELinux status

```
getenforce
# Run with SELinux label
docker run --security-opt
label=type:container_t ...
```

5 User Namespaces

- Maps container user IDs to different IDs on the host.
- UID 0 inside the container is not root on the host.
- Helps reduce the impact of container breakouts.

Enable user namespaces (docker daemon config)

```
"usersns-remap": "default"
```

6 UID 0 Inside a Container

- Running as root (UID 0) in a container does not mean full root on the host.
- Namespace and capability restrictions limit what it can do.
- Still dangerous if combined with misconfigurations (e.g. privileged mode, mounted Docker socket).

7 Privileged Containers

- `--privileged` gives almost full access to the host.
- Disables many security restrictions (capabilities, seccomp, AppArmor/SELinux).
- Should be avoided in production unless absolutely necessary.

Running a privileged container is dangerous
docker run --privileged ...

8 Device Access

- Containers can access host devices (e.g. `/dev/sda`, GPUs) if explicitly allowed.
- Use `--device` or device cgroups.
- Device access increases the risk of host compromise.

Give access to a device

```
docker run --device
/dev/sda:/dev/sda ...
```

9 Read-Only Filesystems

- Use `--read-only` to mount the container filesystem as read-only.
- Prevents writes to the image filesystem (except for tmpfs or mounted volumes).
- Reduces the impact of a compromise.

Run with read-only root filesystem
docker run --read-only ...

10 no-new-privileges

- Prevents processes from gaining more privileges than their parent.
- Stops `setuid/setgid` binaries from escalating privileges.
- Recommended for better security.

Use no-new-privileges
docker run --security-opt
no-new-privileges ...

11 Docker Socket Exposure

- Mounting `/var/run/docker.sock` gives the container full control over the Docker daemon.
- Effectively allows root access to the host.
- Avoid exposing the Docker socket to containers you do not fully trust.

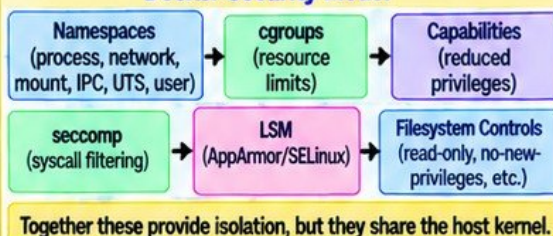
12 Kernel Sharing

- Containers share the host kernel (unlike VMs).
- A kernel vulnerability can affect all containers and the host.
- This is why containers do not provide the same isolation boundary as VMs.

13 Container Escape Risk

- Escapes can happen through:
- Kernel vulnerabilities
- Misconfigurations (privileged, Docker socket)
- Insecure capabilities or devices
- Weak LSM/seccomp profiles
- Always follow the principle of least privilege.

Docker Security Model



14 Key Takeaways

- Containers provide process isolation, not the same isolation as VMs.
- Understand what actually restricts a container.
- Avoid privileged containers.
- Use least privilege (drop capabilities, read-only, no-new-privileges).
- Be careful with Docker socket access.
- Combine multiple layers of security.
- Containers are powerful, but not a security boundary on their own.

Use
Least Privilege.
Stay Secure.
Build Safely.

15 Real-World Example



- A container running as root with the Docker socket mounted can create a new privileged container and gain full access to the host.
- Always review your container security settings.

16 Remember



"Containers isolate processes, but they do not create a separate kernel."

Understand
The Layers.
See The
Big Picture.
Think Like A
Senior Engineer.

containerd, runc and the OCI Runtime Model

Separate Docker from the technologies underneath it.

1 The OCI Standards

- OCI (Open Container Initiative) defines open standards for containers.
- OCI Image Specification:** standard format for container images (layers, manifest, config).
- OCI Runtime Specification:** how a container should run (process, filesystem, namespaces, cgroups, etc).
- OCI Distribution Specification:** how images are stored and distributed in registries.

These standards allow different tools to work together.

2 containerd

- A high-performance container runtime and image manager.
- Manages image downloads, storage, container lifecycle, snapshots, and calls the OCI runtime (e.g. runc).
- Used by Docker (since Docker 1.11) and by Kubernetes (directly).
- Runs as a daemon (containerd).

```
# Check containerd version
containerd --version

# List running containers
ctr containers ls
```

3 runc

- A lightweight OCI-compliant runtime.
- Creates and runs containers based on the OCI runtime specification.
- Handles low-level operations like namespaces, cgroups and filesystem setup.
- Not responsible for image management (that's containerd).
- Other OCI runtimes exist (e.g. crun, kata, gVisor).

```
# Check runc version
runc --version
```

4 Runtime Bundle

- A directory containing everything runc needs to start a container.
- Includes:
 - config.json** (OCI runtime spec)
 - root filesystem** (unpacked image)
 - optional files (hooks, etc).
- Docker/containerd create this bundle automatically.

Example bundle structure

```
bundle/
├── config.json
├── rootfs/
├── bin/
├── etc/
└── ...
```

5 config.json

- A JSON file that follows the OCI runtime specification.
- Defines:
 - process (command, env, user)
 - root filesystem
 - Linux namespaces
 - cgroups (resource limits)
 - mounts
 - hooks (prestart, poststop)

```
# Example: view config.json
cat bundle/config.json
```

6 Root Filesystem

- The unpacked image layers that the container will use as its filesystem.
- Provided in the runtime bundle (e.g. bundle/rootfs).
- Can be read-only or read-write (overlay with a writable layer).
- Contains /bin, /etc, /usr, etc from the image.

7 Lifecycle Operations

- containerd calls runc with lifecycle commands:
 - create** – create the container (sets up bundle)
 - start** – start the container process (PID 1)
 - exec** – run a new process inside a running container
 - kill** – send a signal (e.g. SIGTERM)
 - delete** – clean up resources (namespaces, cgroups, etc)

8 containerd-shim

- A lightweight process that sits between containerd and the OCI runtime.
- Keeps the container running even if containerd (or Docker daemon) restarts.
- Handles stdio, exit status and reaping the container process.
- Each container has its own shim process.

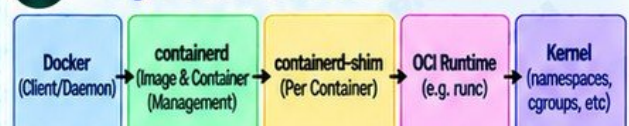
9 Daemon Independence

- Once a container is started, the shim keeps it running independently.
- If the Docker daemon or containerd dies, containers continue to run.
- This improves reliability and allows container management tools to restart without stopping containers.

10 Runtime Replacement

- Docker and containerd can use different OCI runtimes (not just runc).
- Examples: runc, crun, kata-containers (virtualized), gVisor (sandboxed).
- As long as the runtime follows the OCI spec, it can be plugged in.
- This flexibility is why the OCI standard is important.

11 High-Level Architecture



Docker → containerd → shim → OCI Runtime → Kernel
Each layer has a clear responsibility.

12 Real-World Example

- docker run nginx
- Docker sends request to containerd.
- containerd downloads image (if needed).
- containerd creates a runtime bundle.
- containerd calls runc (via shim) to create and start the container.
- runc sets up namespaces, cgroups and filesystem, then starts the process.
- container runs with its own shim, independent of the Docker daemon.

13 Key Takeaways

- ✓ OCI standards make the container ecosystem open and portable.
- ✓ Docker is just one tool on top of containerd and runc.
- ✓ containerd manages images and container lifecycle.
- ✓ runc (or another OCI runtime) actually runs the container.
- ✓ containerd-shim keeps containers running independently.
- ✓ Understanding this boundary makes Kubernetes and other container runtimes easier to understand.
- ✓ You can replace the runtime as long as it follows the OCI spec.

14 Senior Insight



Understanding the Docker → containerd → OCI runtime boundary makes Kubernetes, container-runtime architecture and troubleshooting much easier to reason about at a senior level.

15 Useful Commands

```
# containerd
containerd --version
ctr images ls
ctr containers ls
```

```
# runc
runc --version
```

Learn
The Foundations.
Go Deeper.
Build Real Skills.

16 Remember



"Containers provide process isolation, but they do not create the same isolation boundary as independent virtual machines."

Learn
Build
Share
Grow

Debugging a Container From the Host Down

A senior investigation workflow.

Find The Real Problem. Fix With Confidence. Think Like A Senior.



Scenario: "The application container is running, but users cannot reach it."

Investigate in layers, from the application down to the external network. Identify the failing layer before changing configuration.

1

Application Layer

- Is the application running correctly?
- Is it listening on the expected port?
- Check application logs for errors.

```
docker logs <container>
docker exec -it <container> <app-check>
```

2

Process Layer

- Is the application process running?
- Is it bound to the correct interface (0.0.0.0 vs 127.0.0.1)?
- Check open ports inside the container.

```
docker top <container>
docker exec -it <container> ss -tln
```

3

Socket Layer

- Is the port open and listening?
- Is it TCP or UDP?
- Is it bound to the right address?

```
docker exec -it <container> ss -tln
docker exec -it <container> netstat -tulpn
```

4

Container Namespace

- Check network namespace settings.
- Verify routes and DNS.
- Use nsenter for deeper inspection.

```
docker inspect <container>
nsenter -t <pid> -n ip addr
nsenter -t <pid> -n ip route
```

5

Container Interface

- Check the container's eth0 interface.
- Verify IP address and state.
- Confirm it can reach other containers.

```
docker exec -it <container> ip addr
```

6

veth Pair

- Find the veth pair on the host.
- Check that it is UP and passing traffic.
- Look for dropped packets.

```
ip link | grep veth
ethtool -S <veth> # check errors
```

7

Bridge (docker0)

- Check the Docker bridge interface.
- Verify connected containers.
- Check bridge settings and iptables rules.

```
ip addr show docker0
brctl show # or: bridge link
docker network inspect <network>
```

8

Host Routing

- Check the host routing table.
- Verify the route to the container subnet.
- Confirm the host can reach the container IP.

```
ip route
ip route get <container_ip>
ping <container_ip>
```

9

NAT / Firewall

- Check iptables or nftables rules.
- Verify DNAT and masquerading.
- Look for blocked traffic.

```
sudo iptables -t nat -L -n -v
sudo iptables -L -n -v
sudo nft list ruleset
```

10

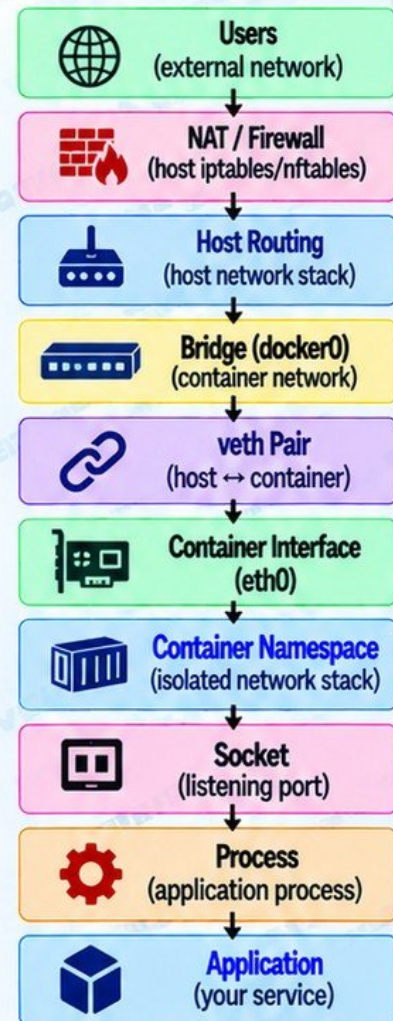
External Network

- Test from an external machine.
- Check security groups (cloud) or firewall rules.
- Verify DNS and public IP (if applicable).

```
curl http://<host-ip>:<port>
traceroute <host-ip> dig <your-domain>
```

11

Layered Investigation Flow



Investigate from the top down. Stop at the failing layer.

12

Evidence Tools (Quick Reference)

Tool	What it helps you check
docker inspect	Container configuration, network settings
docker logs	Application logs
docker stats	Resource usage (CPU, memory, network)
docker top	Running processes
ss	Open sockets and listening ports
ip	Network interfaces, routes, links
nsenter	Enter container namespaces
journalctl	Docker daemon and system logs
dmesg	Kernel messages (network, firewall, errors)

13

Key Takeaways

- Investigate in layers, from application to external network.
- Collect evidence at each layer.
- Identify the failing layer before making changes.
- Use the right tool for the right layer.
- Containers share the host kernel, so host firewall and routing rules matter.
- Don't assume the application is the problem.
- Document your findings.
- This workflow works for Docker, Kubernetes and other container platforms.

14

Common Issues

- Application listening only on 127.0.0.1
- Wrong container port or host port
- Firewall blocking the port
- Missing DNAT or masquerading
- Container on the wrong network
- Veth interface down
- Incorrect routing
- Security group blocking traffic (cloud)
- Application crash or high resource usage
- DNS misconfiguration

Learn
Build
Share
Grow

Production Incident, One Container Failure

From Symptom to Kernel

Real Incidents.
Real Skills.
Real Engineers.
Go Beyond
the Surface.

1

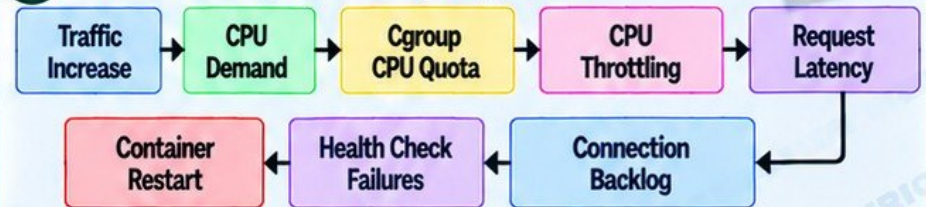
Incident Scenario

A production API container experiences rising latency, intermittent timeouts, and eventually restarts.

- Users report slow responses.
- Some requests time out.
- Health checks start failing.
- The container eventually restarts.

2

Failure Chain (What Actually Happened)



3

Initial Symptoms

- Increasing API latency (p95).
- Intermittent 504 timeouts.
- Health check failures.
- Container restarts in logs.
- Users report degraded performance.

4

Blast Radius

- Affects API service only (initially).
- Downstream services experience timeouts.
- No data loss, but degraded user experience.
- Other containers on the host are healthy.

5

Recent Changes

- New marketing campaign increased traffic.
- No recent application code change.
- Container resource limits were set last week.
- No host updates.

6

Host vs Container Evidence

Container Evidence

- docker logs
- docker stats
- docker inspect
- Container metrics (CPU, memory)
- Application logs
- Health check logs

Host Evidence

- top / htop
- docker stats (all)
- journalctl
- dmesg
- cgroup metrics
- Network stats (ss, netstat)
- System load

7

CPU Throttling Metrics

- Cgroup CPU quota limits the container's CPU usage.
- When demand exceeds quota, the kernel throttles the container.
- This causes increased latency even though CPU usage may look normal.

```
# Check CPU throttling
cat /sys/fs/cgroup/cpu.stat
# or
docker stats <container>
```

8

Network Symptoms

- Increasing connection queue (backlog).
- More connections in SYN_RECV state.
- Timeouts for new requests.
- Not a network outage, but application slow to respond.

```
# Check connections
ss -s
ss -tanp | grep <container_port>
```

9

Misleading Signals

- CPU usage shows 70% (looks fine).
- Memory is normal.
- No errors in application logs.
- Network looks healthy.
- Container restarts may look like an application bug (but the real issue is CPU throttling).

Always check lower layers (kernel / cgroups) when the symptoms do not match the obvious cause.

10

Hypothesis and Root Cause

- **Hypothesis:** increased traffic caused higher CPU demand.
- **Evidence:** high cpu throttling metrics in cgroup.
- **Root cause:** container CPU quota too low for the increased load, causing CPU throttling, which led to request latency, connection backlog, health check failures, and container restarts.

11

Immediate Mitigation

- Increase CPU limit for the container.
- Scale out (more replicas).
- Temporarily reduce traffic if needed.

```
docker update --cpus=2 <container>
```

12

Verification

- Latency returned to normal.
- No more timeouts.
- Container stable (no restarts).
- CPU throttling metrics dropped.

13

Permanent Remediation

- Right-size CPU requests and limits.
- Implement auto-scaling.
- Run load tests before campaigns.
- Review resource limits regularly.

14

Alerting Improvement

- Add alerts for CPU throttling (cgroup metrics).
- Monitor request latency and error rates.
- Alert on container restarts.
- Set SLOs and dashboards.

15

Capacity Planning

- Use historical traffic data.
- Plan for peak events (campaigns, launches).
- Set proper CPU and memory requests/limits.
- Use auto-scaling and load testing.

16

Prevention

- Regular load testing.
- Monitor cgroup metrics.
- Use reasonable resource limits.
- Implement auto-scaling.
- Run game-day exercises.
- Have runbooks for common scenarios.

17

Final Senior Lesson



Docker troubleshooting should move through the abstraction layers until the evidence reveals where Docker ends and the Linux kernel begins.

Containers are powerful, but the kernel is the final authority.

Learn
Build
Share
Grow

VERIQTA

REAL ENGINEERING KNOWLEDGE



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Same
Notes.
Bigger
Engineers.